Agenda:

1. Poobab.
   Relation with Data Analytics
2. Traditional
3. Theoritical
4. Conditional Poob

Data Analytics

Data ⟶ Info? ×
↓
Raw

Process of analyzing data to understand what is happening, why, what is likely to happen next.

Well informed decisions

→ Using data instead of guesswork

How data analytics relates of Probs and Stats

Data is uncertain and incomplete

Probs and Stats are the tools that help us handle this uncertainity

Probs :

⇒ How likely is something to happen.

Stats : • Summarize data (mean median, variance)
• Compare groups
• Draw conclusion from this sample

Likelihood -

Will the student pass an exam?
Will the server crash today?
Will it rain tomorrow?

Quantitative
Objective
Defensible

→ Stats
→ Hypo testing
→ Confidence intervals
→ Regression
→ Classification
→ ML models
→ Risk Analysis
→ Forecasting.

Python

numpy
pandas

store outcomes
count "
estimate prob.
simulate random events

Vocab

Experiment Any process that yield outcomes

Ex Tossing a coin

Rolling a dice

Checking daily attendance

Represented in python: loops, fun^s, simulation

Outcome Single result of an experiment

Head

4

Absent

Represented in python: String
Integer

Sample Space All possible outcomes of an exp

Coin → { H, T }

Dice → { 1, 2, 3, 4, 5, 6 }

Represented in python: list
set

["H", "T"]

[1, 2, 3, 4, 5, 6]

Event Subset of the sample space

Even no. getting rolled = {2, 4, 6}

Getting head = {H}

Represented in python: Filtering using cond^n

Prob. of an event:

Random Variable Assign numbers to outcomes

Dice → 1 to 6

Traditional Prob Ideal conditions

Equal likelihood

Dice ✓
Weather ✗
Customer decision ✗

Real data is never balanced

actions = ["buy", "not buy", "buy"]

Prob comes from freq, not assumption

→ All outcomes should be known

In analytics, New outcomes appear
Data keeps growing

logs = ["click", "scroll", "new tab", "exit", "click"]

Convert the logs list to set

set (logs)

→ T. Prob assumes that prob. fixed

In real life, Prob change with time & / with data

Theoretical Prob Vs Empirical Prob

↓ Calculated using maths

↓ Actual data

no ex

Prob of rolling 6 on a dice = $\frac{1}{6}$

P(E)

clicks = 120

views = 1000

logs = ["click", "no click", "click", "click"]

logs.count ("click") / len (logs)

Numpy spefically designed for numbers

→ fast numerical arrays

→ built in maths | stats func

→ Random number generator

→ Foundation for ML, Pandas

NumPy Array Homogeneous

Stored in conti.

Optimized for math op

	List	NumPy
Mean	x	.mean()
Var,	x	.var()
Math op	Loop	Vectorize
Random	Hard	Built in

When to use numpy array

Data is numeric
Speed
Math / Stats

Not use

Labelled
Tabular

```
Probability of an event
P(E)=favourable outcomes/total outcomes
```

```
dice_space=[1,2,3,4,5,6] # sample space

#event: favourable outcome (getting an even number)
even_event=[2,4,6]

# count of favourable outcomes
favourable=len(even_event)

# total possible outcomes
total=len(dice_space)

prob_even=favourable/total
print(prob_even)
```

```
0.5
```

```
import random
x=random.choice([1,2,3,4,5,6])
print(x)
```

```
2
```

```
import random
heads=0
n=1000

for i in range(n):
  if random.choice(["H","T"])=="H":
    heads+=1

print(heads/n)
```

```
0.517
```

```
# Basics of python

x=10 # int
y=0.5 # float
name="A" # string
a='u' #string


# lists -> sample space, data storage, observations

# loops -> experiment, simulation, counting
# conditions -> events, filtering
```

```
# numpy

# lists -> general containers they not maths tools
-> millions of values -> slow of large data
-> no built in maths/stats functions
-> require a loop
-> memory inefficient
```

```
data=[1,2,3,4,5,567,8]

# mean
mean=sum(data) / len(data)
print(mean)
```

```
84.28571428571429
```

```
import numpy as np

# create an array
arr=np.array([10,20,30,40])
print(arr)
```

```
[10 20 30 40]
```

```
# using a loop
arr=[1,2,3,4,5]
result=[]

for x in arr:
  result.append(x+10)
print(result)
```

```
[11, 12, 13, 14, 15]
```

```
import numpy as np
arr= np.array([1,2,3,4])

result=arr+10
print(result)
```

```
[11 12 13 14]
```

```
# random number generate
tosses=np.random.choice(["H","T"],size=1000)

print(tosses[:20])
```

```
['T' 'H' 'H' 'T' 'H' 'T' 'H' 'H' 'T' 'H' 'H' 'T' 'T' 'H' 'H' 'T' 'H' 'H'
 'T' 'H']
```

```
# empirical probab

heads_count=n
```

```
0.52
```